GUARDIAN MCP — Multi-Step Attack & Information Leak Detector

HACKURITY 2026 · Problem statement: verify official PS number/title · Team: [add team name and four members]

1. Problem and scope

MCP-connected AI agents can invoke tools to read information and perform actions. A single call may appear legitimate while a sequence of calls can create an unsafe outcome—for example, attempting to transfer information after reading a permitted report. GUARDIAN MCP is a runtime security gateway intended to inspect tool requests, apply policy checks, identify suspicious sequences, and block prohibited operations before execution. Its scope is a local demonstration using controlled tools and synthetic sample data; it does not claim complete protection for all agents or production deployments.

2. Approach and architecture

The gateway evaluates each request against configured policy and tool restrictions, input/intent checks, information-flow controls, multi-step attack detection, and rule-based risk scoring. It returns decisions such as ALLOW or BLOCK; approval-required handling should be claimed only where demonstrated. Security events can be correlated into an AttackTrace and written to a SHA-256 hash-linked audit log. A Streamlit dashboard and Windows notifications provide visibility where enabled.

```
Request / test harness → MCP security gateway → policy + intent + input validation + information-flow +
  sequence detection + risk scoring → ALLOW / BLOCK / APPROVAL_REQUIRED → permitted tool execution →
                                audit log / AttackTrace / dashboard
```

3. Core deliverables, status and where to see them

Core deliverable	Status*	Where to verify
Runtime security gateway	Working*	app/gateway/runtime.py; tests/test_gateway.py
Least-privilege policy engine	Working*	app/policy/engine.py; tests/test_policy.py
IntentGuard / attack classification	Working*	app/security/intent_guard.py, attack_classifier.py; matching tests
Information-flow controls	Working*	app/security/information_flow.py; information-flow tests
Multi-step detection / AttackTrace	Working*	app/security/attack_sequence.py, attack_trace.py; matching tests
Risk scoring / audit integrity	Working*	app/security/risk_scoring.py; app/audit/logger.py; matching tests
Approval, dashboard, notifications	Partial*	app/security/human_approval.py; dashboard/app.py; app/security/desktop_notifications.py. Verify live.

*Provisional statuses: update them to match what is demonstrable in the exact submitted commit. A source file alone is not proof.

GUARDIAN MCP — Technical Write-up (continued)

4. Stretch objectives

Potential stretch items are AttackTrace correlation, numerical risk scoring, SHA-256 hash-chain verification, human approval, and desktop notifications. Mark each Working only after showing it in the running build; otherwise mark Partial or Not done. Evidence should include visible decisions, traces, verification output, approval results, or notifications.

5. Threat or user model

Threats include unauthorized tool calls, prohibited operations, unsafe arguments, and multi-step actions that may expose protected information. Intended users are developers and security reviewers who need an enforcement point and an inspectable event trail. The demo assumes relevant tool calls pass through the gateway and policy configuration is correct.

6. Assumptions

The demonstration uses a local Python environment, controlled sample tools, and non-sensitive sample data. Detection and risk scores are rule-based. All claims and test counts must correspond to the exact commit under review.

7. Evidence: numbers and how measured

An earlier project note reported 66 passing tests; this has not been independently verified for the current commit. Before submission, run `python -m pytest -v` and record the actual pass/fail count. Replay: (1) a permitted request returning ALLOW; (2) a prohibited request returning BLOCK with `executed: False`; and (3) a multi-step scenario where an external-send attempt is blocked. Verify audit-chain integrity using the project's actual verification command and record the output. Do not claim detection rates or accuracy without a defined test set and calculation.

8. Known limitations and next steps

Rule-based checks may miss novel attacks or create false positives. Sequence detection only sees events available to the gateway. Hash-linked logs can help reveal edits but do not alone ensure secure storage or authenticity if an attacker controls the logger. Approval state may be in-memory; confirm persistence behavior. Next steps: evaluate false positives/negatives on a documented dataset, persist approvals and logs securely, test that all tool calls are mediated, and harden deployment before production use.

Submission verification: Replace the official PS number/title and team placeholders. Confirm each status and evidence path against the submitted build. The previously reported test count is not presented as a verified result.